❤ Like | Save | Share



Eric Hu

JavaScript Promises





Creating a promise

A promise is created with the `Promise` constructor, which receives `resolve` and `reject` callbacks.

```js
const promise = new Promise((resolve, reject) => {
  // async work here
});
```

Resolving a promise

Call `resolve` when the operation succeeds.

```js
const success = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("Success!");
  }, 1000);
});

success.then((message) => console.log(message)); // "Success!"
```

Rejecting a promise

Call `reject` when the operation fails.

```js
const failure = new Promise((resolve, reject) => {
  reject(new Error("Something went wrong"));
});

failure.catch((error) => console.error(error.message));
// "Something went wrong"
```

Promise states

State	Description
`pending`	Initial state, neither fulfilled nor rejected.
`fulfilled`	The operation completed successfully.
`rejected`	The operation failed.

then / catch / finally

Handle the outcome of a promise.

```js
fetch("https://api.example.com/user")
  .then((response) => response.json())
  .then((data) => console.log(data))
  .catch((error) => console.error(error))
  .finally(() => console.log("Done"));
```

`then` is called when the promise is fulfilled.

`catch` is called when the promise is rejected.

`finally` is called regardless of the outcome.

Chaining

Each `then` returns a new promise, allowing you to chain operations.

```js
function double(value) {
  return value * 2;
}

Promise.resolve(5)
  .then(double)
  .then(double)
  .then((result) => console.log(result)); // 20
```

Promise.all

Waits for all promises to fulfill. Rejects immediately if any promise rejects.

```js
const a = Promise.resolve(1);
const b = Promise.resolve(2);
const c = Promise.resolve(3);

Promise.all([a, b, c]).then((values) =>
console.log(values)); // [1, 2, 3]
```

Promise.race

Returns the result of the first promise that settles, either fulfilled or rejected.

```js
const fast = new Promise((resolve) => setTimeout(resolve, 100, "fast"));
const slow = new Promise((resolve) => setTimeout(resolve, 500, "slow"));

Promise.race([fast, slow]).then((result) => console.log(result)); // "fast"
```

Promise.allSettled

Waits for all promises to settle, regardless of outcome.

```js
const promises = [Promise.resolve("ok"), Promise.reject("error")];

Promise.allSettled(promises).then((results) => console.log(results));
// [
//   { status: "fulfilled", value: "ok" },
//   { status: "rejected", reason: "error" }
// ]
```

Promise.any

Returns the first fulfilled promise. Only rejects if all promises reject.

```js
const a = Promise.reject("fail");
const b = Promise.resolve("success");

Promise.any([a, b]).then((result) =>
console.log(result)); // "success"
```

Async / await

`async` and `await` is a cleaner syntax for working with promises.

```js
async function getUser(id) {
  try {
    const response = await fetch(`/api/users/${id}`);
    const user = await response.json();
    return user;
  } catch (error) {
    console.error(error);
  }
}
```

Await multiple promises in parallel.

```js
async function loadDashboard() {
  const [user, posts] = await Promise.all([
    fetch("/api/user").then((r) => r.json()),
    fetch("/api/posts").then((r) => r.json()),
  ]);

  return { user, posts };
}
```

Full-Stack Developer Roadmap AI

From HTML & CSS to working with AI models, all in one structured roadmap.

www.thedevspace.io